

Generative KI für 3D-Medizinbildgebung mittels Diffusion Models

Generative AI for 3D Medical Imaging using Diffusion Models

Studienarbeit

vorgelegt von

Sven Beller

2026

Matrikelnummer, Kurs

5102447, TINF23

Betreuer

Malte Tölle

Sperrvermerk

TODO: Anpassen

Die vorliegende Arbeit beinhaltet interne vertrauliche Informationen des Unternehmens ZIEHL-ABEGG SE. Sie ist nur für die Beteiligten am Begutachtungs- und Evaluationsprozess bestimmt. Die Weitergabe des Inhalts der Arbeit im Gesamten oder in Teilen sowie das Anfertigen von Kopien oder Abschriften - auch in digitaler Form - sind grundsätzlich untersagt. Ausnahmen bedürfen der schriftlichen Genehmigung des Unternehmens ZIEHL-ABEGG SE.

Dieser Sperrvermerk gilt unbefristet.

Künzelsau, 1. Januar 2025

Sven Beller

Erklärung

Ich, **Name**, versichere hiermit, dass ich meine **Hier T3_2000/T3_3100 etc** mit dem Thema **Hier Thema** selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre zudem, dass ich generative Systeme (Text-, Bild-, Codeerstellung etc.) bei der Erstellung der vorliegenden wissenschaftlichen Arbeit in der nachfolgend beschriebenen Art und dem nachfolgend beschriebenen Umfang verwendet habe.

Ort, Datum

Name

Genutzter Funktionsumfang

Themenerfassung und Strukturierung (Aufgabenstellung/Präzisierung, Forschungsansatz, Gliederung)

Produkt/Tool	Beschreibung / Verwendungsweise
Tool	Ideenfindung und Grobgliederung der Arbeit. Beispiel: Welche Themen existieren über Thema?

Quellenrecherche, -auswahl und -auswertung (durch AI gefundene Quellen; Vorgehen der weiteren Recherche)

Produkt/Tool	Beschreibung / Verwendungsweise
Tool	Vorschläge relevanter Publikationen, anschließende manuelle Validierung. Beispiel: Welche Bücher behandeln das Thema Thema?

Formale Gestaltung, insbesondere Sprache (Prompts/Eingaben; Textgenerierung, Korrektur, Paraphrasieren, Übersetzen, kreatives Schreiben)

Produkt/Tool	Beschreibung / Verwendungsweise
Tool	Grammatik-/Stilkorrektur einzelner Absätze; keine inhaltliche Neuerstellung. Beispiel: Ist der folgende Satz grammatikalisch richtig?

Abstract

This is the abstract in English.

Zusammenfassung

Dies ist die Zusammenfassung auf Deutsch.

Inhaltsverzeichnis

Abbildungsverzeichnis	IX
Tabellenverzeichnis	X
Nomenklatur	XII
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	1
1.3 Vorgehensweise	1
2 Medizinische Bildgebung	2
2.1 Bildgebende Verfahren	2
2.2 NIfTI	2
3 Künstliche Intelligenz	3
3.1 Definition Künstliche Intelligenz	3
3.1.1 Definition Intelligenz	4
3.1.2 Schwache und Starke KI	4
3.2 Maschinelles Lernen	5
3.2.1 Diskriminative Modelle	5
3.3 Generative Modelle	5
3.4 Diffusion Model	6
3.5 Test	6
3.5.1 Markov-Ketten	6
3.5.2 Noise-Schedule	7
3.5.3 Vorwärtsprozess	7
3.5.4 Rückwärtsprozess	8
3.5.5 Sampling	9
3.5.6 Conditional Diffusion Models	10
3.6 Convolutional Neural Network	10

3.7	U-Net	11
3.8	Einsatzmöglichkeiten in der Medizin	13
3.8.1	GANs	13
3.8.2	VAE	14
4	Datenqualität und ethische Aspekte	15
4.1	Kriterien für realistische Bilddaten	15
4.2	Risiken und Grenzen synthetischer Daten	15
4.3	Datenschutz	15
4.4	Bewertungsmetriken	15
4.4.1	Fréchet Inception Distance	15
4.4.2	SSIM	16
4.4.3	PSNR	17
4.4.4	LPIPS	17
4.4.5		17
5	Implementierung und experimenteller Aufbau	18
5.1	Entwicklungsumgebung und Hardware	18
5.2	Datenvorverarbeitung	19
5.3	Modellarchitektur	19
5.4	Baseline-Konfiguration	20
5.5	Trainingsverfahren und Hyperparameter	20
5.6	Ablationsstudien	21
5.7	Aufgezeichnete Werte	22
6	Ergebnis	23
6.1	Trainingsergebnisse	23
6.1.1	Modellkonfigurationen	23
6.1.2	Wichtigste Erkenntnisse	26
6.1.3	Detaillierte Metriken-Analyse	27
6.1.4	Fazit	29
7	Schlussfolgerung	30
7.1	Fazit	30
7.2	Ausblick	30

Literaturverzeichnis	XV
A Anhang A	XIX

Abbildungsverzeichnis

2.1	NifTI-Header [3, S. 76]	2
3.1	Markov-Kette erster Ordnung [17, S. 609]	6
3.2	Darstellung der Vorwärts- und Rückwärtsprozesse	7
3.3	Convolutional [20, S. 6]	10
3.4	CNN	11
3.5	U-Net Architektur [25, S. 2]	12
4.1	Darstellung der Fréchet Inception Distance (FID)-Berechnung [29, S. 130] . .	16
4.2	Darstellung der SSIM (SSIM)-Berechnung [32, S. 5]	17

Tabellenverzeichnis

Nomenklatur

Lateinische Buchstaben

A	m^2	Fläche
$\vec{a}$	m/s^2	Beschleunigung
a	m/s	Schallgeschwindigkeit
c_p	J/(kgK)	spezifische isobare Wärmekapazität
c_v	J/(kgK)	spezifische isochore Wärmekapazität
E	J	innere Energie
e	J/kg	spezifische innere Energie
g	m/s^2	Erdbeschleunigung
H	J	Enthalpie
h	J/kg	spezifische Enthalpie
k	J/K	Boltzmann-Konstante $(k = 1,3807 \cdot 10^{-23})$
m	kg	Masse
N_A	mol^{-1}	Avogadro-Konstante $(N_A = 6,0021318 \cdot 10^{23})$
$\vec{n}$	-	nach außen gerichteter Normaleneinheitsvektor
n	-	Polytrophenexponent
P	J/s	Leistung
p	N/m^2	Druck
Q	J	Wärme
$\dot{Q}$	J/s	Wärmestrom
q	J/kg	bezogene Wärme
$\dot{q}$	J/(kg s)	bezogener Wärmestrom
R	J/(kgK)	spezielle Gaskonstante
R_m	J/(molK)	universelle Gaskonstante $(R_m = 8,31441)$
T	K	Temperatur
t	s	Zeit
u, v, w	m/s	Geschwindigkeitskomponenten im kartesischen Koordinatensystem
$\vec{V}$	m/s	Geschwindigkeitsvektor
V	m^3	Volumen
v	m^3/kg	spezifisches Volumen
W	J	Arbeit
x, y, z	m	kartesische Koordinaten
z	m	Höhe

Griechische Buchstaben

η	Pa s	dynamische Viskosität (Scherviskosität)
κ	-	Isentropenexponent
ν	m ² /s	kinematische Viskosität
γ	-	Polytropenverhältnis
ρ	kg/m ³	Dichte
τ_w	N/m ²	Wandschubspannung
$\vec{\omega}$	s ⁻¹	Winkelgeschwindigkeit

Tiefgestellte Indizes

0	Ruhegröße
n	normal
t	tangential

Hochgestellte Indizes

*	kritischer Zustand (LAVAL-Zustand)
---	------------------------------------

Dimensionslose Kennzahlen

Ma	:= $\ \vec{V}\ /a$	Mach-Zahl
Ma*	:= $\ \vec{V}\ /a^*$	kritische Mach-Zahl
Re	:= $\ \vec{V}\ L/\nu$	Reynolds-Zahl

Operatoren, Funktionen und Symbole

Abkürzungen

KI	Künstliche Intelligenz
NLP	Natural Language Processing
DGM	Deep generative model
NIfTI	Neuroimaging Informatics Technology Initiative
NIH	National Institutes of Health
CNN	Convolutional Neural Network
FID	Fréchet Inception Distance
IS	Inception score
GANs	Generative Adversarial Networks

IQMs	Image Quality Metrics
FR	full-reference
RR	reduced-reference
NR	no-reference
SSIM	SSIM
PSNR	Peak Signal-to-Noise Ratio
MSE	Mean squared Error
ReLU	Rectified Linear Unit
ANN	Artificial Neural Networks
GAN	Generative Adversary Network
VAE	Variational Autoencoder Network
DDPM	Diffusion Denoising Probabilistic Model
DDIM	Diffusion Denoising Implicit Model

1 Einleitung

TODO: neu schreiben/bisschen abändern

In der Medizin werden mithilfe von 3D-Bildgebung bessere Diagnosen von Krankheiten und genauere Eingriffe durchgeführt. Dabei bringt die steigende Entwicklung von generativer KI neue Möglichkeiten für medizinische Bilddaten.

Diffusion Models haben sich als leistungsfähige Methode zur realistischen Bildsynthese etabliert. In der datengetriebenen Medizin besteht ein wachsender Bedarf an hochwertigen, vielfältigen und datenschutzkonformen Bilddaten, insbesondere für das Training von KI-Systemen. Dieses Projekt widmet sich der Entwicklung eines Diffusion Models zur Generierung synthetischer 3D-medizinischer Volumendaten, die für Forschungszwecke, Augmentierung und Trainingsprozesse genutzt werden können.

1.1 Problemstellung

Für generative KI-Modelle besteht ein wachsender Bedarf an Daten, um ein Model darauf zu trainieren. Allerdings ist die Verfügbarkeit dieser Daten eingeschränkt durch Datenschutzbestimmungen und an der Bildakquise von seltenen Krankheiten. Durch diese Limitierung besteht eine Erschwerung des Trainieren solcher Modelle. Es besteht daher ein Bedarf an synthetischen aber realistischen Bilddaten, die sowohl qualitativ und ethisch vertretbar sind.

1.2 Zielsetzung

Ziel dieser Studienarbeit ist die Entwicklung eines generativen KI-Systems, das mittels Diffusion Models realistische 3D-medizinische Bilddaten erzeugt.

1.3 Vorgehensweise

2 Medizinische Bildgebung

2.1 Bildgebende Verfahren

2.2 NIfTI

Das Neuroimaging Informatics Technology Initiative (NIfTI)-Format (.nii, .nii.gz) wurde in den frühen 2000er Jahren vom National Institutes of Health (NIH) entwickelt und hat sich als Standardformat in der neuroimaging-basierten Forschung etabliert. Es ist speziell für die Verarbeitung volumetrischer 3D-Bilddaten ausgelegt und daher geeignet für beispielsweise MRT-Gehirnuntersuchungen [1, S. 203]. NIfTI ist ein Rasterformat, das Daten in dreidimensionaler Form als Voxel speichert also als Pixel mit einer definierten Breite, Höhe und Tiefe [2, S. 4]. Dabei werden in den ersten drei Dimensionen die räumlichen Koordinaten x , y und z gespeichert, während die vierte Dimension für den Zeitpunkt t reserviert ist [3, S. 76]. Der Header einer NIfTI-Datei umfasst im .nii-Format 352 Bytes [3, S. 76]. In Abb. 2.1 ist ein Beispiel-Header zu sehen, welcher zeigt, welche Metadaten gespeichert werden. So ist beispielsweise im Parameter ImageSize die Auflösung des MRT-Scans abzulesen, in diesem Fall 256x156x256. Im Gegensatz zu Formaten

Parameters	Value
Filename	group00001_T1w_CSF_probability.nii
Filesize	40894816
Description	'DAD210715'
ImageSize	[256 156 256]
xPixelDimensions	[1 1.3000 1]
Datatype	'single'
BitsPerPixel	32
SpaceUnits	'Millimeter'
TimeUnits	'Second'
MultiplicativeScaling	1

Abb. 2.1: NIfTI-Header [3, S. 76]

wie JPEG, das eine verlustbehaftete Kompression verwendet und dadurch relevante Bildinformationen beeinträchtigen kann, unterstützt NIfTI eine verlustfreie Kompression [1, S. 203].

3 Künstliche Intelligenz

TODO: Titel gegebenenfalls anpassen **Schreibe: Schreiben worüber in diesem Kapitel behandelt wird**

3.1 Defintion Künstliche Intelligenz

Der Begriff Künstliche Intelligenz (KI) kann je nach Anwendungsgebiet eine andere Definition haben. Ein Buch beschreibt die KI als ein Forschungsgebiet [4, S. 5]. Jedoch wird die KI in dieser Studienarbeit beschrieben als eine Methode, dass Computern die Intelligenz des Menschen vorgibt und somit menschenähnlich zu denken [5, S. 3] [6, S. 42].

TODO: Agenten nochmal neu definieren. Siehe S. 17 Knowledge Science - Grundlagen

3.1.1 Defintion Intelligenz

Die Intelligenz kann als eine Sammlung von Eigenschaften verstanden werden und es ist das Ziel dass diese Eigenschaften nachgebaut werden. Wissenschaftler in der KI haben die folgenden Eigenschaften zu der Intelligenz definiert: [7, S. 1 f.]

- **Wahrnehmung:** Die Beobachtung seiner Umwelt mithilfe von Sensoren.
- **Schlussfolgern:** Aus bekannten Informationen rationale Entscheidungen ziehen auch bei Unsicherheit.
- **Handeln:** Aus bekannten Informationen gezielt handeln und Werkzeuge nutzen oder entwickeln.
- **Planung und Problemlösung:** Schritte planen, um ein Ziel zu erreichen oder ein Problem zu lösen.
- **Kommunikation:** Mit anderen Systemen oder Menschen kommunizieren.
- **Anpassungsfähigkeit und Lernen:** Aus Erfahrungen lernen und sich an neue Situationen anpassen.
- **Autonomie:** Eigene Ziele setzen und selbst entscheiden, wie man sie erreicht.
- **Kreativität:** Neue Wege zur Lösung von Problemen finden.
- **Reflexion und Bewusstheit:** Über eigene Entscheidungen und Prozesse reflektieren.
- **Ästhetik:** Bei Entscheidungen auch ästhetische Aspekte berücksichtigen.
- **Organisation:** Mit anderen zusammenarbeiten und sich abstimmen.

Schreibe: Über die Punkte angehen, wie diese die Intelligenz beschreibe **TODO: Bearbeiten** Anschließend zu der Intelligenz zählt das rationale Denken. Beim rationalen Denken wird auf Basis des bekannten Informationen auch mit Unsicherheit, die bestmögliche Entscheidung getroffen. Das System soll auch auf diese Entscheidungen handeln und selbstständig weiterlernen. Ein solches System wird als *intelligenter Agent* bezeichnet [8, S. 4].

3.1.2 Schwache und Starke KI

Wie intelligent eine KI ist, wird unter Wissenschaftler zwischen einer *schwachen (weak)* / *beschränkten (narrow)* und *starken (strong)* / *allgemeine (general)* KI unterschieden [9, S. 1]. Eine schwache KI ist spezifisch für eine Domäne entwickelt, die die menschliche Intelligenz dort simuliert [9, S. 1] [10, S. 7]. Diese können ein oder wenige spezifische Probleme sehr gut lösen. Allerdings kann die schwache KI bekannte Lösungsmuster auf

neue Probleme nicht in andere Domänen übertragen [10, S. 7]. Momentan können alle KI-Systeme als eine schwache KI bewertet werden, da sie jeweils spezifisch zu ihrer Domäne sind. Eine beispiel Domäne ist die Natural Language Processing (NLP) [9, S. 1]. Die schwache KI wird noch mal in zwei weitere Kategorien verfasst und unterscheiden sich dabei, mit welcher Methode sie trainiert werden. Diese sind die *symbolische KI* und *nicht-symbolische KI*. In der symbolischen KI wird diese mithilfe von Deduktion trainiert. Heißt auf logische Regeln. Während die nicht-symbolische KI mit Induktion also anhand von vorhandene Daten lernt [5, S. 4 f.]. Eine starke KI hingegen besitzt die Fähigkeit die menschliche Intelligenz zu emulieren. Dies bedeutet, dass sie selbständig fungiert und somit auch selbstbewusst ist. Dabei kann die starke KI Probleme lösen, ohne davor dafür trainiert zu sein [10, S. 7] [11, S. 17]. Dies würde dann zu einer technologischen Singularität führen und einer Superintelligenz [9, S. 1] [10, S. 7] [11, S. 17]. Eine technologische Singularität ist der Punkt, an dem die KI nicht mehr vom Menschen kontrolliert werden kann [11, S. 18]. Allerdings gibt bis jetzt keine starke KI und somit ein theoretisches Konzept. Es wird jedoch in Science-Fiction-Werke angewendet [9, S. 1] [11, S. 17 f.] [5, S. 4].
TODO: Neu durchlesen

3.2 Maschinelles Lernen

Zu diesen Methoden gehört das maschinelle Lernen, wobei der Computer selbständig Wissen generiert aus Erfahrungen. Dabei wird der Computer durch Beispiele trainiert worraus dieser dann Muster abgeleitet werden. Diese Muster werden dann auf die jeweilige Situation angewendet [6, S. 42]. **Schreibe: Was ist Intelligenz Schreibe: Je weniger Eigenschaften gebracuth werden desto besser können Ergebnisse sein**

3.2.1 Diskriminative Modelle

3.3 Generative Modelle

Generative Modelle hingegen sind in der Lage neue synthetische Daten zu generieren. Sie lernen, wie die Daten strukturiert sind und können durch das Erlernen dieser Struktur neue Daten generieren, die dieser Struktur folgen [12, S. 112].

Generative Model werden auch als Deep generative model (DGM)s bezeichnet. Unter ein DGM wird ein neuronales Netzwerk verstanden, dass die Wahrscheinlichkeitsverteilung von hoch dimensionalen Daten, wie Bilder, Text oder Audio, lernt [13, S. 16]. Sie besitzen eine hohe Anzahl von versteckten Schichten aus Neuronen, um das Erlernen der hochdimensionalen Datenverteilungen zu ermöglichen. Allerdings sind sie sehr rechenaufwendig und die Performanz ist stark abhängig von *Hyperparametern*. Hyperparameter beziehen sich auf Einstellungen, die vor dem Training festgelegt werden und den Lernprozess steuern. Dazu gehören die Netzwerkarchitektur, die Wahl der Zielfunktion, Regularisierungstechniken sowie Trainingsalgorithmen [14, S. 2].

3.4 Diffusion Model

Ein *Diffusion Model* oder auch *Diffusions-Wahrscheinlichkeitsmodell* ist ein generatives Modell, das zur Klasse der latenten Variablenmodelle gehört [15, S. 153] [12, S. 114]. Das grundlegende Prinzip besteht darin, einen Diffusionsprozess umzukehren, bei dem schrittweise Rauschen zu den Daten hinzugefügt wird. Durch die Umkehrung dieses Prozesses kann das Modell aus verrauschten Signalen Daten rekonstruieren und so neue Daten generieren. Dafür wird eine parametrisierte Markov-Kette mittels Variationsinferenz trainiert, deren Übergänge lernen, den Diffusionsprozess umzukehren [15, S. 153] [16, S. 2].

3.5 Test

Sehr viel Text

3.5.1 Markov-Ketten

Eine Markov-Kette ist ein stochastischer Prozess, bei dem der zukünftige Zustand ausschließlich vom aktuellen Zustand abhängt und nicht von der Vergangenheit. Diese Eigenschaft wird als *Gedächtnislosigkeit* bezeichnet. Für eine Sequenz von Zufallsvariablen $x_0, x_1, \dots, x_T$ lässt sich dies ausdrücken als:

$$p(x_t | x_0, \dots, x_{t-1}) = p(x_t | x_{t-1}), \quad t = 1, \dots, T \quad (3.1)$$

Hierbei bezeichnet t den Zeitschritt und x_t den Zustand zum Zeitpunkt t . Eine solche Markov-Kette, bei der der aktuelle Zustand nur vom unmittelbar vorhergehenden abhängt, wird als Markov-Kette *erster Ordnung* bezeichnet [17, S. 609]. In Abb. 3.1 wird diese graphisch dargestellt. Aufgrund der Markov-Eigenschaft kann die gemeinsame Wahr-

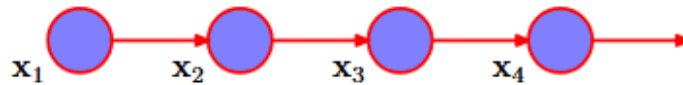


Abb. 3.1: Markov-Kette erster Ordnung [17, S. 609]

scheinlichkeit der gesamten Sequenz $x_{1:T}$ gegeben x_0 als Produkt der einzelnen Übergangswahrscheinlichkeiten faktorisiert werden:

$$p(x_{1:T} | x_0) = \prod_{t=1}^T p(x_t | x_{t-1}) \quad (3.2)$$

Diese Faktorisierung ist für Diffusionsmodelle von zentraler Bedeutung [15, S. 153 f.] [17, S. 608 f.]. Die Gedächtnislosigkeit der Markov-Kette erweist sich als besonders geeignet für Diffusionsprozesse, da jeder Schritt sowohl beim Hinzufügen als auch beim Entfernen von Rauschen ausschließlich vom aktuellen Zustand abhängt.

3.5.2 Noise-Schedule

Der *Noise Schedule* definiert eine Sequenz von vorgegebene Werten $\beta_1, \beta_2, \dots, \beta_T$ mit $\beta_t \in (0,1)$, die steuern wie viel Rauschen in jedem Schritt t hinzugefügt wird. Aus diesen werden zwei Werte definiert [13, S. 43 f.] [15, S. 155 f.]:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{k=1}^t \alpha_k \quad (3.3)$$

Dabei gibt α_t an, welcher Anteil des Signals aus dem vorherigen Schritt erhalten bleibt. Hingegen tut $\bar{\alpha}_t$ das kumulative Produkt über alle bisherigen Schritte darstellen. Mit steigendem t nimmt $\bar{\alpha}_t$ monoton ab, sodass das ursprüngliche Signal zunehmend durch Rauschen überschrieben wird [13, S. 44]. Heißt, wenn für $t \rightarrow T$ gilt $\bar{\alpha}_T \approx 0$, was bedeutet, dass das Originalbild nahezu vollständig zerstört ist.

Die Wahl des Noise Schedulers beeinflusst die Qualität der generierten Daten. Der *lineare Schedule*, der von Ho et al. [16, S. 5] verwendet wird, definiert β_t als lineare Interpolation zwischen einem Start- und Endwert:

$$\beta_t = \beta_1 + \frac{t-1}{T-1}(\beta_T - \beta_1) \quad (3.4)$$

Allerdings wurde beobachtet dass der lineare Schedule in den späten Schritten zu aggressiv rauscht. Nichol und Dhariwal [18, S. 4] verwenden daher einen *Cosine Schedule*, der $\bar{\alpha}_t$ direkt über eine Kosinusfunktion definiert:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad \text{mit} \quad f(t) = \cos\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)^2 \quad (3.5)$$

wobei s ein kleiner Offset-Parameter ist, der verhindert, dass β_t in der Nähe von $t = 0$ zu klein wird. Der Cosine Schedule führt zu einem gleichmäßigeren Rauschverlauf und verbessert die Bildqualität insbesondere bei niedrigen Auflösungen.

TODO: Vllt. Bild hinzufügen aus Nichol?

3.5.3 Vorwärtsprozess

Die Abb. 3.2 stellt den Vorwärts- und Rückwärtsprozess an einem Bild dar. Der Vorwärts-

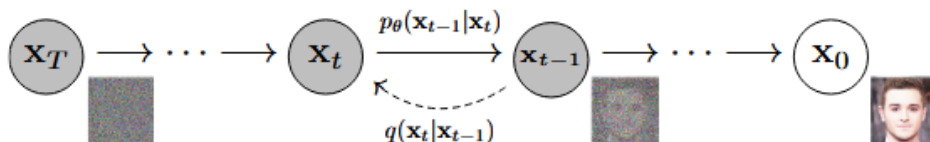


Abb. 3.2: Darstellung der Vorwärts- und Rückwärtsprozesse

prozess (engl. *Forward Process*) ist fest vorgegeben, wird nicht trainiert und dient als Encoder. Er fügt einem Signal x_0 schrittweise Gaußsches Rauschen hinzu bis am Ende reines Rauschen übrig ist. Die Übergangsverteilung für einen einzelnen Schritt ist definiert als [15, S. 155 f.]:

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right) \quad (3.6)$$

Hierbei wird das Signal aus dem vorherigen Schritt mit dem Faktor $\sqrt{1 - \beta_t}$ skaliert und Gaußsches Rauschen mit Varianz β_t hinzugefügt. Wobei eben $0 < \beta_1 < \beta_T < 1$ gilt. Da jeder Schritt nur vom vorherigen Zustand abhängt, kann die gemeinsame Verteilung des gesamten Vorwärtsprozesses gemäß der Markov-Eigenschaft faktorisiert werden mit [15, S. 156] [16, S. 2]:

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}) \quad (3.7)$$

Dies bedeutet, dass ein Bild x_0 nach T Schritten vollständig zerstört wird und das Ergebnis x_T reinem Gaußschem Rauschen $\mathcal{N}(0, I)$ entspricht. Ein Vorteil des Vorwärtsprozesses ist, dass x_t nicht iterativ berechnet werden muss. Mithilfe der Definitionen $\alpha_t = 1 - \beta_t$ und $\bar{\alpha}_t = \prod_{k=1}^t \alpha_k$ lässt sich x_t direkt aus dem Originalbild x_0 berechnen [15, S. 156] [16, S. 2] [13, S. 44 f.]:

$$q(x_t | x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I\right) \quad (3.8)$$

Daraus ergibt sich die folgende Formel [15, S. 156] [16, S. 2] [13, S. 44 f.]:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (3.9)$$

Dadurch kann direkt zu einem beliebigen Rauschzustand x_t gesprungen werden ohne alle Zwischenschritte $x_0, x_1, \dots, x_{t-1}$ durchlaufen zu müssen. Dies ermöglicht dann ein effizientes, da zufällige Zeitschritte t gesampelt werden können [16, S. 2 f.].

3.5.4 Rückwärtsprozess

Der Rückwärtsprozess (engl. *Reverse Process*) ist das Gegenstück zum Vorwärtsprozess und dient als lernbarer Decoder. Er startet bei reinem Rauschen $x_T \sim \mathcal{N}(0, I)$ und entfernt in jedem Schritt einen Teil des Rauschens, bis ein sinnvolles Datensample x_0 rekonstruiert ist. Auch dieser Prozess wird als Markov-Kette dargestellt [16, S. 2]:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t) \quad (3.10)$$

Da die wahren Übergangswahrscheinlichkeiten $q(x_{t-1} | x_t)$ von der unbekannten Datenverteilung abhängen, wird jeder Rückwärtsschritt durch ein neuronales Netz mit Parametern θ approximiert [16, S. 2]:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}\left(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I\right) \quad (3.11)$$

Parametrisierung des Mittelwerts

Anstatt den Mittelwert μ_θ direkt zu schätzen, wird das Netzwerk darauf trainiert, das hinzugefügte Rauschen $\epsilon_\theta(x_t, t)$ vorherzusagen. Aus der Gl. (3.9) des Vorwärtsprozesses lässt sich der Mittelwert dann wie folgt berechnen [16, S. 4]

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \quad (3.12)$$

Diese Parametrisierung ist vorteilhaft, da die Vorhersage des Rauschens stabiler zu optimieren ist als die direkte Mittelwertschätzung. Damit ergibt sich der vollständige Sampling-Schritt im Rückwärtsprozess als [16, S. 4]:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) \quad (3.13)$$

Die Varianz σ_t^2 wird üblicherweise als fester Wert gesetzt, wobei $\sigma_t^2 = \beta_t$ oder $\sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$ verwendet wird und für z entweder $z \sim \mathcal{N}(0, I)$ oder $z = 0$ gilt, wenn es für den letzten Schritt $t = 1$ gilt [16, S. 4].

3.5.5 Sampling

Das Sampling beschreibt den Prozess der Datengenerierung, bei dem von reinem Rauschen $x_T \sim \mathcal{N}(0, I)$ schrittweise der Rückwärtsprozess durchlaufen wird [16, S. 4]. Die Wahl des Sampling-Verfahrens beeinflusst sowohl die Qualität der generierten Daten als auch die benötigte Rechenzeit.

DDPM-Sampling

Das Diffusion Denoising Probabilistic Model (DDPM)-Sampling folgt direkt dem stochastischen Rückwärtsprozess aus Gl. (3.13). Es durchläuft alle T Zeitschritte (meistens $T = 1000$ [16, S. 5]) und fügt bei jedem Schritt stochastisches Rauschen hinzu:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) \quad (3.14)$$

Dieses Verfahren liefert qualitativ hochwertige Ergebnisse, ist jedoch aufgrund der T erforderlichen Netzwerkauswertungen rechenintensiv [19, S. 3].

DDIM-Sampling

Im Gegensatz zum DDPM-Sampling müssen beim Diffusion Denoising Implicit Model (DDIM)-Sampling nicht alle T Schritte durchlaufen werden. Dies wird dadurch ermöglicht, dass der Vorwärtsprozess auch als nicht-Markov-Prozess interpretiert werden

kann, was zu einer deterministischen Sampling-Prozedur führt [19, S. 4 f.]. Das DDIM-Sampling wird wie folgt beschrieben:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(x_t, t) + \sigma_t z \quad (3.15)$$

Für $\sigma_t = 0$ wird der Prozess deterministisch, das heißt derselbe Startpunkt x_T erzeugt immer dasselbe Ergebnis [19, S. 5]. In der Praxis reichen häufig 50 bis 100 Schritte statt für $T = 1000$, was eine 10- bis 50-fache Beschleunigung bei vergleichbarer Bildqualität ermöglicht [19, S. 7].

3.5.6 Conditional Diffusion Models

3.6 Convolutional Neural Network

Convolutional Neural Network (CNN) ist eine Klasse neuronaler Netzwerke, die speziell für die Verarbeitung von Bilddaten entwickelt wurde [20, S. 1]. Dies geschieht durch sogenannte Faltungsschichten (engl. *convolutional layers*), die mithilfe von Filtern lokale Muster in der Datenstruktur erkennen. Diese Merkmalsextraktion ermöglicht es dem Modell, komplexe Strukturen mit hoher Genauigkeit zu identifizieren [21, S. 1170].

Die Faltung (engl. *convolution*) ist eine mathematische Operation, bei der ein Faltungskern (engl. *kernel*) schrittweise über ein Bild geschoben wird. An jeder Position wird das elementweise Produkt zwischen Kernel und dem überdeckten Bildausschnitt aufsummiert. Formal ergibt sich für ein zweidimensionales Bild f mit Kernel k :

$$g(x, y) = \sum_i \sum_j f(x + i, y + j) \cdot k(i, j) \quad (3.16)$$

Das Ergebnis ist eine *Feature Map*, die beschreibt, wie stark ein bestimmtes Muster, wie eine Kante oder Textur an jeder Bildposition vorhanden ist [22, S. 14]. In Abb. 3.4 wird ein

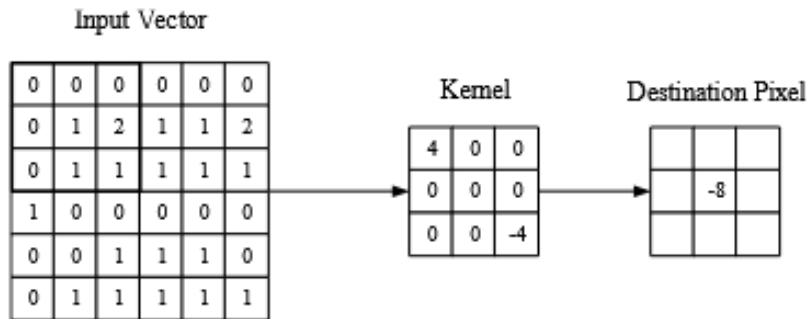


Abb. 3.3: Convolutional [20, S. 6]

CNN dargestellt. Die Architektur basiert auf drei Schichttypen. In der Faltungsschicht, wird wie der Name es bezeichnet die Faltung durchgeführt und Merkmale extrahiert [20,

S. 5 f.]. Parallel zu den Faltungsoperationen folgt eine Aktivierungsfunktion (ReLU). Diese führt eine Nichtlinearität hinzu und reduziert das Vanishing-Gradient-Problem [21, S. 1169]. Als Aktivierungsfunktion wird standardmäßig die Rectified Linear Unit (ReLU) mit $f(x) = \max(0, x)$ eingesetzt, die ein schnelleres Training als traditionelle Funktionen wie $\tanh(x)$ ermöglicht [23, S. 3].

Darauf folgt die *Pooling-Schicht*. Diese reduziert die räumliche Auflösung, typischerweise durch Max-Pooling mit 2×2 -Fenstern, wodurch die Aktivierungskarte auf 25% ihrer Größe verkleinert wird. Damit werden die Anzahl der Parameter sowie die Rechenkomplexität des Modells schrittweise reduziert [20, S. 8].

Die *Fully-Connected-Schicht* am Ende des Netzwerks verbindet jeden Neuron direkt mit allen Neuronen der benachbarten Schichten, analog zur klassischen Artificial Neural Networks (ANN)-Architektur. Dadurch werden die extrahierten Merkmale zu Klassenswahrscheinlichkeiten verarbeitet [20, S. 8]. Im Gegensatz zu traditionellen ANN, bei de-

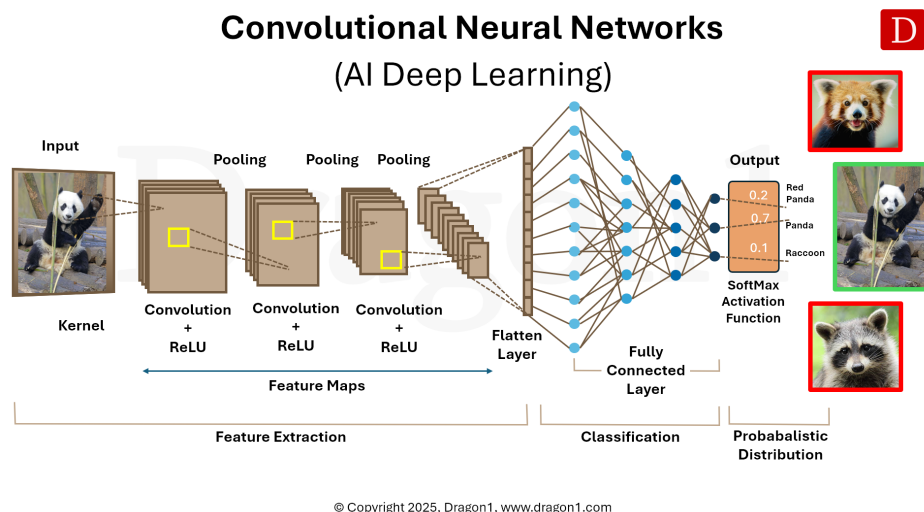


Abb. 3.4: CNN

nen jedes Neuron mit allen Neuronen der vorherigen Schicht verbunden ist, verbinden sich Neuronen in CNN nur mit einem kleinen lokalen Bereich der Eingabe [20, S. 6]. Diese lokale Konnektivität reduziert die Anzahl der Parameter erheblich und macht das Training effizienter [20, S. 2–3].

Eine typische CNN-Architektur stapelt mehrere Faltungs- und Pooling-Schichten, wobei frühe Schichten einfache Merkmale und tiefere Schichten zunehmend komplexe, abstrakte Repräsentationen lernen [20, S. 8–9]. Diese hierarchische Merkmalsextraktion bildet die Grundlage für den Encoder-Pfad in Architekturen wie U-Net [24, S. 6].

3.7 U-Net

U-Net ist eine CNN-Architektur, die für die biomedizinische Bildsegmentierung entwickelt wurde. Trotz ihres ursprünglichen Anwendungsbereiches dient die Architektur ei-

ne zentrale Rolle im Denoising-Schritt in Diffusionsmodelle [25, S. 2]. In Abb. 3.5 wird

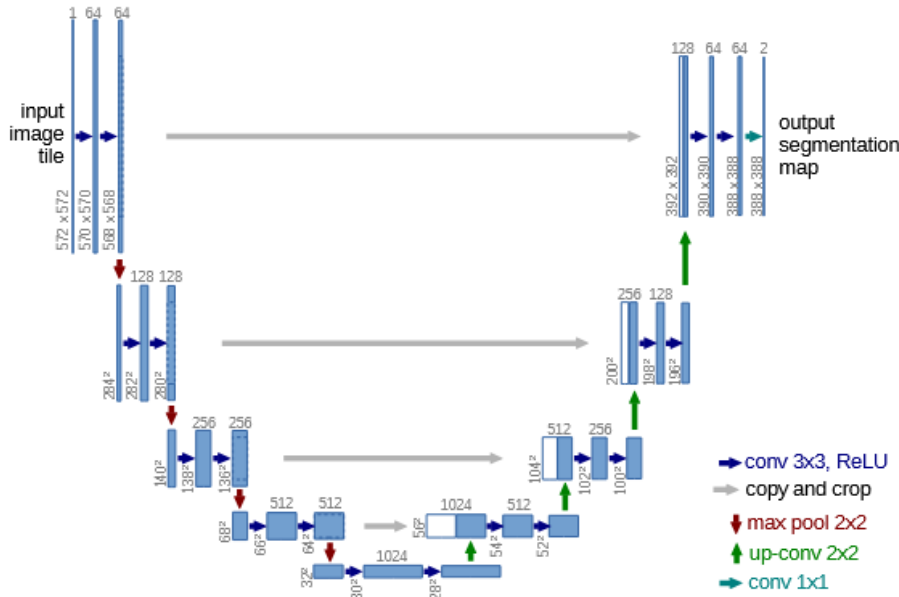


Abb. 3.5: U-Net Architektur [25, S. 2]

die Architektur dargestellt. Das Grundprinzip von U-Net basiert auf einer symmetrischen Encoder-Decoder-Struktur. Der Encoder, auch als *Contracting Path* bezeichnet, folgt dem Prinzip eines typischen CNNs und besteht aus wiederholten Blöcken von zwei 3x3-Faltungen (unpadded) mit anschließender ReLU-Aktivierung sowie einem 2x2 Max-Pooling mit Schrittweite 2 zum Downsampling [25, S. 4]. Mit jeder Downsampling-Stufe verdoppelt sich die Anzahl der Feature-Kanäle, während die räumliche Auflösung schrittweise reduziert wird [25, S. 4]. Dieser Prozess ermöglicht es dem Netzwerk, zunehmend abstrakte semantische Merkmale zu extrahieren und hierarchische Repräsentationen zu erlernen [24, S. 6].

Der Decoder, auch *Expansive Path* genannt, spiegelt diese Struktur symmetrisch und besteht aus Upsampling-Schritten mittels 2x2-Up-Convolutions, gefolgt von zwei 3x3-Faltungen mit ReLU [25, S. 4]. Das zentrale Merkmal der U-Net-Architektur sind die sogenannten Skip Connections, dargestellt als graue Pfeile in Abb. 3.5. Diese verbinden korrespondierende Encoder- und Decoder-Ebenen direkt miteinander, indem die Feature-Maps konkateniert werden [25, S. 4] [24, S. 9]. Dadurch können feinkörnige räumliche Informationen, die beim Downsampling verloren gehen, im Decoder wiederhergestellt werden. Die finale Schicht verwendet eine 1x1-Faltung, um die Feature-Vektoren auf die gewünschte Anzahl von Klassen abzubilden [25, S. 4].

Mathematisch lässt sich ein einzelner Faltungsschritt beschreiben als:

$$\mathbf{x}^{(l+1)} = \sigma\left(W^{(l)} * \mathbf{x}^{(l)} + b^{(l)}\right), \quad (3.17)$$

wobei $W^{(l)}$ den Faltungsfilter, $\mathbf{x}^{(l)}$ die Eingabe-Feature-Map, $b^{(l)}$ den Bias und σ die ReLU-Aktivierungsfunktion der l -ten Schicht bezeichnen. Die Skip Connection im Decoder lässt sich formal als $\mathcal{F}_x^{(l)} = \text{Concat}(\mathcal{E}_x^{(l)}, \mathcal{D}_x^{(l)})$ beschreiben, wobei $\mathcal{E}_x^{(l)}$ und $\mathcal{D}_x^{(l)}$ die

Feature-Maps von Encoder und Decoder auf Ebene l repräsentieren und $\mathcal{F}_x^{(l)}$ die fusionierte Feature-Map darstellt [24, S. 9]. Diese Konkatenation ermöglicht es, hochauflösende strukturelle Details aus dem Encoder mit den abstrakten semantischen Repräsentationen des Decoders zu verbinden.

Im Kontext von Diffusionsmodellen wird U-Net eingesetzt, um bei jedem Denoising-Schritt das Rauschen $\epsilon_\theta(\mathbf{x}_t, t)$ zu schätzen, wobei der Zeitschritt t als zusätzliche Konditionierungsinformation in die Architektur einfließt. Die Vielseitigkeit von U-Net zeigt sich nicht nur in der Anwendung über verschiedene Domänen hinweg, sondern auch in der Integration in unterschiedliche Modelltypen wie Diffusionsmodelle, Generative Adversarial Networks (GANs) und autoregressive Architekturen. Durch die Flexibilität der symmetrischen Encoder-Decoder-Struktur mit Skip Connections hat sich U-Net als Standardarchitektur in modernen Diffusionsmodellen für verschiedene Datenmodalitäten etabliert, darunter Bild-, Audio-, Video- und 3D-Generierung [24, S. 2].

Die U-Net-Architektur bietet mehrere Vorteile für generative Aufgaben. Die hierarchische Encoder-Decoder-Struktur ermöglicht eine Merkmalsextraktion auf mehreren Auflösungsebenen, wodurch sowohl globaler Kontext als auch lokale Details erfasst werden können. Die Skip Connections behalten detaillierte räumliche, temporale und strukturelle Details, die bei herkömmlichen Autoencoder-Architekturen durch das Downsampling verloren gehen würden. Darüber hinaus besitzt U-Net eine hohe Recheneffizienz, da die faltungsbasierten Operationen parallelisiert werden können und im Gegensatz zu Transformer-Modellen linear mit der Eingabegröße skalieren. Weitere Stärken umfassen die Robustheit gegenüber verrauschten oder unvollständigen Eingabedaten sowie die Flexibilität zur Integration von Attention-Mechanismen und Residual Connections [24, S. 34 ff.].

Trotz dieser Vorteile weist U-Net auch Limitationen auf. Eine wesentliche Einschränkung ist die begrenzte Fähigkeit von globalem Kontext, da die faltungsbasierte Struktur primär auf lokale räumliche Merkmale spezialisiert ist. Bei hochauflösenden generativen Aufgaben steigt der Speicherbedarf erheblich, während das rezeptive Feld der Faltungsschichten proportional kleiner wird. Zudem kann die Generalisierungsfähigkeit bei domänenfremden Daten oder multimodalen Aufgaben wie Text-zu-Bild-Generierung eingeschränkt sein, da U-Net auf pixelbasierte Transformationen ausgelegt ist. Schließlich operiert U-Net wie viele tiefe neuronale Architekturen als Black-Box, was die Interpretierbarkeit der gelernten Repräsentationen erschwert [24, S. 32 f.] [24, S. 40 f.].

3.8 Einsatzmöglichkeiten in der Medizin

3.8.1 GANs

Generative Adversary Network (GAN)s bestehen aus zwei neuronalen Netzwerken, die gegeneinander trainiert werden: einem *Generator* G und einem *Diskriminator* D . Der Generator erzeugt aus einem zufälligen Rauschen $z \sim p_z(z)$ synthetische Daten $G(z)$, während der Diskriminator lernt, echte Daten $x \sim p_{data}(x)$ von gefälschten zu unter-

scheiden. Dieses Wettbewerbsprinzip wird als *Minimax-Spiel* bezeichnet und durch folgende Zielfunktion beschrieben:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (3.18)$$

Der Diskriminator D versucht, $D(x) \approx 1$ für echte und $D(G(z)) \approx 0$ für gefälschte Daten zu erreichen. Der Generator G hingegen versucht, den Diskriminator zu täuschen, sodass $D(G(z)) \approx 1$. Im Idealfall konvergiert das Training zu einem Nash-Gleichgewicht, bei dem G die wahre Datenverteilung p_{data} approximiert.

3.8.2 VAE

Ein Variational Autoencoder Network (VAE) ist ein generatives Modell, das Daten in einen komprimierten *latenten Raum* z kodiert und aus diesem neue Daten rekonstruiert. Im Gegensatz zu klassischen Autoencodern kodiert der VAE nicht einen festen Punkt, sondern eine Wahrscheinlichkeitsverteilung $q_\phi(z | x) = \mathcal{N}(\mu, \sigma^2 I)$, aus der dann Samples gezogen werden. Das Training minimiert die folgende Verlustfunktion, den *Evidence Lower Bound* (ELBO):

$$\mathcal{L}_{VAE} = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{Rekonstruktionsfehler}} - \underbrace{D_{KL}(q_\phi(z | x) \parallel p(z))}_{\text{Regularisierung}} \quad (3.19)$$

Der erste Term minimiert den Rekonstruktionsfehler – das Modell soll die Eingabe x möglichst gut wiederherstellen. Der zweite Term, die *KL-Divergenz*, zwingt den latenten Raum nah an eine Standardnormalverteilung $\mathcal{N}(0, I)$ zu bleiben, sodass neue Daten durch einfaches Sampling aus $p(z)$ generiert werden können.

4 Datenqualität und ethische Aspekte

4.1 Kriterien für realistische Bilddaten

4.2 Risiken und Grenzen synthetischer Daten

4.3 Datenschutz

4.4 Bewertungsmetriken

Um die Qualität von durch generativen Modelle erstellten Bildern objektiv zu beurteilen, kommen sogenannte Image Quality Metrics (IQMs) zum Einsatz. IQMs erlauben einen objektiven Rahmen zur Bewertung von Eigenschaften wie Auflösung, Rauschen und Artefakte. Gegenüber subjektiven Bewertungen durch Menschen haben IQMs den Vorteil, dass sie standardisiert, quantifizierbar und reproduzierbar sind und somit einen systematischen Vergleich verschiedener generativer Modelle ermöglicht. IQMs lassen sich in drei verschiedenen Kategorien einordnen: full-reference (FR), reduced-reference (RR) und no-reference (NR). Diese Unterscheidung ist im Kontext von Diffusionsmodellen besonders wichtig. Bei Aufgaben wie Bildrekonstruktion oder Denoising, bei denen ein Originalbild als Referenz vorliegt, kommen FR-Metriken wie SSIM oder Peak Signal-to-Noise Ratio (PSNR) zum Einsatz. Bei der freien Text-to-Image-Generierung hingegen existiert kein eindeutiges Referenzbild, weshalb hier häufig auf NR-Metriken oder semantische Maße wie den FID oder den CLIP-Score zurückgegriffen wird [26, S. 1]. **TODO: mehr quelle zu IQM finden**

4.4.1 Fréchet Inception Distance

Die FID ist eine Metrik zur Bewertung der Qualität GANs und wurde 2017 von Heusel et al. eingeführt. Sie stellt eine Weiterentwicklung des Inception score (IS) dar, dessen Schwachpunkt darin besteht, dass er die statistische Verteilung realer Bilder nicht in die Bewertung einbezieht [27, S. A1]. Die Berechnung der FID erfolgt in zwei Schritten. Zunächst werden sowohl reale als auch generierte Bilder durch ein vortrainiertes CNN geleitet [28, S. 2]. Aus einer intermediären Schicht dieses Netzwerks werden abstrakte Merkmalsvektoren gewonnen, die als visuelle Repräsentationen der Bilder dienen [29, S. 130]. Im zweiten Schritt werden diese Merkmale statistisch beschrieben. Für die realen Bilder wird der Mittelwert μ_{data} also der durchschnittliche Merkmalsvektor sowie die Kovarianzmatrix Σ_{data} berechnet, welche beschreibt, wie stark die Merkmale untereinander

variieren. Dasselbe geschieht für die generierten Bilder, die durch μ_g und Σ_g beschrieben werden [29, S. 130] [27, S. A1]. Anschließend wird die Fréchet Distanz zwischen den beiden so beschriebenen Verteilungen berechnet [30, S. 2]:

$$\text{FID} = \|\mu_{data} - \mu_g\|^2 + \text{Tr}\left(\Sigma_{data} + \Sigma_g - 2\left(\Sigma_{data}\Sigma_g\right)^{1/2}\right), \quad (4.1)$$

Die Formel Gl. (4.1) besteht aus zwei Teilen. Der erste Term $\|\mu_{data} - \mu_g\|^2$ misst, wie weit die durchschnittlichen Merkmale realer und generierter Bilder voneinander abweichen. Der zweite Term vergleicht die Kovarianzmatrizen beider Verteilungen und erfasst damit, ob die generierten Bilder eine ähnliche Vielfalt und Struktur aufweisen wie die realen cite [30, S. 2]. In Abb. 4.1 wird der Prozess dargestellt. Der Generator G erzeugt Bilder, die

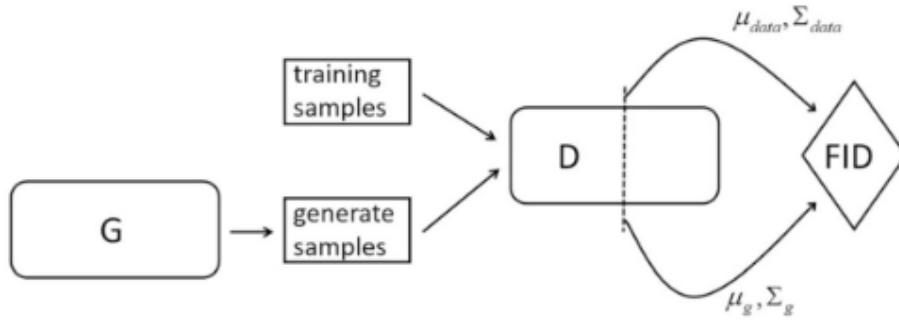


Abb. 4.1: Darstellung der FID-Berechnung [29, S. 130]

zusammen mit den Trainingsbildern durch das CNN D geleitet werden, um die statistischen Kenngrößen μ_{data} , Σ_{data} , μ_g und Σ_g zu extrahieren und daraus den FID-Wert zu berechnen. Ein niedriger FID-Wert zeigt an, dass beide Verteilungen eng beieinanderliegen, was auf hohe Bildqualität und ausreichende Diversität hindeutet [29, S. 130].

4.4.2 SSIM

Der SSIM ist eine Metrik zur Bewertung von Bildqualität, die sich an der menschlichen Wahrnehmung orientiert [31, S. 1131] [26, S. 6]. Im Gegensatz zur Mean squared Error (MSE), der lediglich pixelweise Helligkeitsunterschiede misst, analysiert der SSIM, wie ähnlich zwei Bilder in ihrer Struktur sind also in den Mustern und Abhängigkeiten benachbarter Pixel, die für das menschliche Auge relevant sind [32, S. 1]. Dazu zerlegt SSIM den Bildvergleich in drei Teilkomponenten: Intensität, Kontrast und Struktur. Diese werden getrennt berechnet und anschließend multipliziert [32, S. 5 f.]. Die resultierende Formel lautet [33, S. 2] [32, S. 5] [26, S. 6] [31, S. 1134]:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (4.2)$$

In Gl. (4.2) stehen μ_x und μ_y für die mittleren Helligkeitswerte, σ_x^2 und σ_y^2 für die lokalen Varianzen und σ_{xy} für die Kovarianz also das Maß, wie ähnlich sich die Helligkeitsverläufe der beiden Bilder in einem lokalen Bereich verhalten. Die Konstanten C_1

und C_2 verhindern numerische Instabilitäten bei sehr kleinen Nennerwerten und sind als $C_1 = (K_1 L)^2$ und $C_2 = (K_2 L)^2$ definiert, wobei L den Wertebereich der Pixel angibt und typischerweise $K_1 = 0,01$ sowie $3K_2 = 0,03$ gewählt werden [33, S. 1 f.] [32, S. 5] [26, S. 6] [31, S. 1134]. Der berechnete SSIM-Wert eines Bildes ergibt sich als Mittelwert aller lokal berechneten Fensterwerte und liegt stets zwischen 0 und 1, wobei 1 für eine vollständige Übereinstimmung bedeutet [31, S. 1131]. Jedoch ist der SSIM eine FR-Metrik und benötigt somit ein fehlerfreies Referenzbild [32, S. 1]

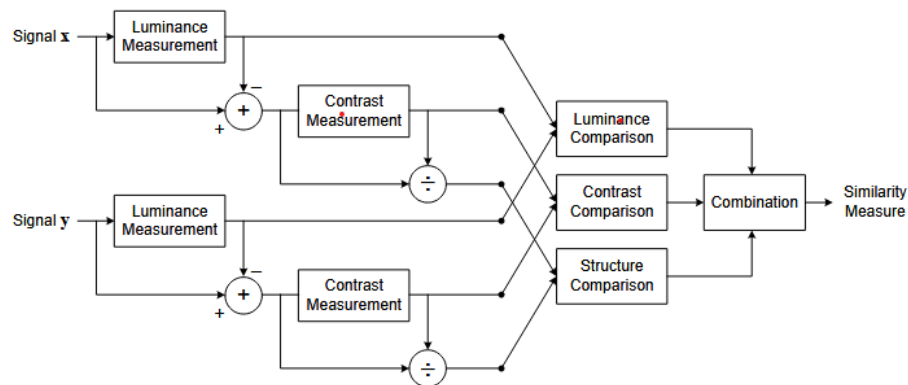


Abb. 4.2: Darstellung der SSIM-Berechnung [32, S. 5]

4.4.3 PSNR

4.4.4 LPIPS

4.4.5

5 Implementierung und experimenteller Aufbau

Das folgende Kapitel beschreibt die praktische Umsetzung der in dieser Arbeit untersuchten Modelle. Zunächst werden die eingesetzten Werkzeuge und die zugrunde liegende Hardware-Konfiguration vorgestellt. Anschließend wird auf die Architektur der einzelnen Modelle sowie die Wahl der jeweiligen Hyperparameter eingegangen und deren Einfluss auf das Trainingsverhalten erläutert.

5.1 Entwicklungsumgebung und Hardware

Das Training der Modelle wurde auf einem lokalen System durchgeführt. Dieses verfügt über eine NVIDIA GeForce GTX 1080 Ti mit 11 GB VRAM, einen AMD Athlon X4 860K Prozessor (4 Kerne, 3,7 GHz), 16 GB Arbeitsspeicher sowie ein Linux-Ubuntu-Desktop-Betriebssystem. Der Einsatz einer dedizierten GPU ermöglicht eine Reduktion der Trainingszeiten gegenüber einer reinen CPU-basierten Berechnung.

Die Implementierung sämtlicher Modelle erfolgte in der Programmiersprache Python. Python besitzt eine umfangreiche Anzahl an Bibliotheken, wie Keras und TensorFlow, für maschinelles Lernen und hat sich als De-facto-Standard in diesem Bereich etabliert [34]. Als Entwicklungsumgebung wurde Visual Studio Code in Kombination mit Jupyter-Notebooks eingesetzt. Die Verwendung von Jupyter-Notebooks ermöglicht eine schrittweise Ausführung einzelner Code-Zellen, was bei der Entwicklung und dem Debugging von Vorteil ist. Für jedes Modell wurde ein separates Notebook angelegt, um eine klare Trennung der Experimente zu gewährleisten.

Für den Modellaufbau wurde das Deep-Learning-Framework PyTorch eingesetzt. Die Diffusionslogik einschließlich des Noise Schedulers, des Sampling-Verfahrens und der Trainingsschleife wurde auf Basis von PyTorch implementiert. Für das Laden und Speichern der medizinischen Bilddaten im NIFTI-Format wurde die Bibliothek nibabel verwendet. Die Visualisierung der Ergebnisse erfolgte mit Matplotlib. Die Implementierung basiert auf Python 3.12.3 und PyTorch 2.7.1 mit CUDA 11.8. Die Wahl der CUDA-Version ergibt sich aus der Tatsache, dass die NVIDIA GeForce GTX 1080 Ti auf der Pascal-Architektur (Compute Capability 6.1) basiert, welche von neueren CUDA-Versionen nicht mehr vollständig unterstützt wird.

5.2 Datenvorverarbeitung

Vor dem Training wurden die MRT-Volumina des BraTS-2023-Datensatzes vorverarbeitet. Die Vorverarbeitung orientiert sich an der Datenverarbeitung von Durrer, Cattin und Wolleb [35, S. 6]. Es umfasst fünf Schritte, die im Folgenden beschrieben werden.

Intensitätsbegrenzung. Die Intensitätswerte werden auf das Intervall zwischen dem 0,5%- und dem 99,5%-Perzentil beschnitten. Dadurch werden extreme Ausreißer entfernt, die etwa durch Aufnahmeartefakte entstehen können, ohne relevante Bildinformationen zu verlieren.

Hintergrundentfernung. Anschließend wird der Hintergrund entfernt, der hauptsächlich aus Luft und leeren Voxeln besteht. Dazu wird die kleinste Bounding Box berechnet, die alle Voxel mit einem Intensitätswert größer null enthält. Das Volumen wird so auf den anatomisch relevanten Bereich zugeschnitten.

Kubisches Padding. Da die Volumina nach dem Cropping unterschiedliche Dimensionen aufweisen, werden sie durch symmetrisches Zero-Padding auf eine kubische Form gebracht. Im Gegensatz zu einer direkten Skalierung bleibt durch das Padding das Seitenverhältnis erhalten und es werden keine geometrischen Verzerrungen eingeführt.

Resizing. Das kubische Volumen wird mittels trilinearer Interpolation auf die Zielauflösung von 32^3 oder 48^3 Voxeln skaliert. Da das Volumen bereits kubisch ist, erfolgt die Skalierung in allen drei Raumrichtungen gleichmäßig.

Normalisierung. Abschließend werden die Intensitätswerte linear auf den Wertebereich $[-1, 1]$ normalisiert. Dies ist die gängige Konvention für Diffusionsmodelle, da der Rauschprozess auf einem standardisierten Wertebereich arbeitet.

5.3 Modellarchitektur

Als Architektur dient ein 3D-UNet, das dem in der DDPM-Literatur etablierten Aufbau folgt Ho, Jain und Abbeel [16]. Die Architektur ist symmetrisch als Encoder-Decoder aufgebaut und besteht aus vier Auflösungsstufen mit Kanalbreiten von 96, 192, 384 und 384. Im Encoder wird die räumliche Auflösung schrittweise halbiert, während der Decoder sie über trilineare Interpolation wieder auf die Eingabegröße bringt. Zwischen den entsprechenden Stufen von Encoder und Decoder werden Skip-Connections eingesetzt, um räumliche Informationen direkt weiterzugeben.

Residualblöcke und Zeiteinbettung Auf jeder Auflösungsstufe befinden sich zwei Residualblöcke, die jeweils aus zwei 3D-Faltungsschichten mit GroupNorm und SiLU-Aktivierung bestehen. Der aktuelle Diffusions-Zeitschritt wird über sinusoidale Positionseinbettungen kodiert und über eine lineare Projektion in jeden Residualblock eingespeist, sodass das Netzwerk sein Verhalten an den jeweiligen Rauschgrad anpassen kann.

Self-Attention. Auf den beiden niedrigsten Auflösungsstufen werden zusätzlich Self-Attention-Schichten eingesetzt, die es dem Netzwerk ermöglichen, globale Abhängigkeiten zwischen räumlich entfernten Regionen zu modellieren. Auf höheren Auflösungen wird darauf verzichtet, da der Speicherbedarf mit der Voxelanzahl ansteigt.

5.4 Baseline-Konfiguration

Die Baseline-Konfiguration vereint mehrere Techniken, die sich in der aktuellen Literatur als Verbesserungen gegenüber dem ursprünglichen DDPM-Ansatz von Ho et al. [ho2020ddpm] etabliert haben. Sie dient als Referenzmodell, gegen das alle weiteren Experimente verglichen werden.

Der Diffusionsprozess verwendet 250 Zeitschritte mit einem Cosine Schedule [nichol2021improved]. Im Vergleich zum linearen Schedule verteilt der Cosine Schedule das Signal-Rausch-Verhältnis gleichmäßiger über die Zeitschritte was insbesondere bei niedrigen Auflösungen zu einer höheren Bildqualität führt [nichol2021improved].

Das Modell wird mit v-Prediction trainiert [salimans2022progressive]. Dabei sagt das Netzwerk weder das Rauschen ϵ noch das ursprüngliche Bild x_0 direkt vorher, sondern eine Kombination aus beiden, die als Velocity v bezeichnet wird. Dieses Vorhersageziel hat sich als numerisch stabiler erwiesen, insbesondere bei niedrigen Signal-Rausch-Verhältnissen [salimans2022progressive].

Für die Gewichtung des Trainingsverlusts wird Min-SNR Weighting mit $\gamma = 5$ eingesetzt [hang2023minsnr]. Diese Strategie begrenzt den Einfluss von Zeitschritten mit hohem Signal-Rausch-Verhältnis auf den Gesamtverlust und sorgt so für ein ausgewogeneres Training über alle Zeitschritte hinweg.

5.5 Trainingsverfahren und Hyperparameter

Alle Modelle werden über 300 Epochen mit dem Adam-Optimizer und einer Lernrate von 2×10^{-4} trainiert. Diese Lernrate entspricht dem in der DDPM-Literatur üblichen Wert [ho2020ddpm]. In den ersten 500 Iterationen wird die Lernrate linear von null auf den Zielwert erhöht (Warmup), um Instabilitäten zu Beginn des Trainings zu vermeiden. Die Batch Size beträgt 10, was dem verfügbaren GPU-Speicher von 11 GB geschuldet ist.

Zur Stabilisierung der Modellgewichte wird Exponential Moving Average (EMA) mit einem Decay-Faktor von 0,9999 eingesetzt [nichol2021improved]. Dabei wird parallel zu

den trainierten Gewichten ein gleitender Durchschnitt geführt, der für die finale Evaluation und das Sampling verwendet wird. Zusätzlich wird Gradient Clipping mit einer maximalen Norm von 1,0 angewendet, um Gradientenexplosionen zu verhindern.

Der Datensatz wird in 90% Trainings- und 10% Validierungsdaten aufgeteilt. Die Aufteilung erfolgt mit einem festen Random Seed, um die Reproduzierbarkeit sicherzustellen. Während des Trainings wird der Validierungsverlust nach jeder Epoche berechnet.

5.6 Ablationsstudien

Um den Einfluss einzelner Komponenten auf die Generierungsqualität zu untersuchen werden ausgehend von der Baseline-Konfiguration gezielt einzelne Variablen verändert. Pro Experiment wird jeweils nur eine Komponente angepasst, um deren isolierten Einfluss bewerten zu können. Die Ablationsstudien werden auf einer Auflösung von 32^3 Voxeln durchgeführt.

Ohne Self-Attention. Die Self-Attention-Schichten auf den niedrigen Auflösungsstufen werden entfernt. Dadurch lässt sich untersuchen, welchen Beitrag die Modellierung globaler Abhängigkeiten zur Bildqualität leistet und ob die lokalen Informationen der Faltungsschichten allein ausreichen.

Linearer Schedule. Der Cosine Schedule wird durch einen linearen Schedule ersetzt, bei dem die Varianz gleichmäßig von $\beta_1 = 0,0001$ bis $\beta_T = 0,02$ ansteigt. Dies entspricht dem ursprünglichen Vorschlag von Ho et al. [ho2020ddpm].

Noise-Prediction (ϵ -Prediction). Anstelle der v-Prediction wird das Modell darauf trainiert, das hinzugefügte Rauschen ϵ direkt vorherzusagen. Dies ist die ursprüngliche DDPM-Formulierung [ho2020ddpm] und dient als Vergleich zur moderneren v-Prediction.

Datenaugmentierung. Die Trainingsdaten werden durch zufällige räumliche Transformationen augmentiert, um die effektive Datensatzgröße zu erhöhen und die Generalisierung zu verbessern.

Erhöhte Auflösung (48^3). Zusätzlich wird die Baseline-Konfiguration auf einer Auflösung von 48^3 Voxeln trainiert, um den Einfluss der räumlichen Auflösung auf die Bildqualität zu untersuchen. Aufgrund der deutlich längeren Trainingszeiten bei höherer Auflösung werden die übrigen Ablationen nur auf 32^3 durchgeführt.

5.7 Aufgezeichnete Werte

Während des Trainings werden für jede Epoche verschiedene Metriken protokolliert, um den Trainingsverlauf nachvollziehen zu können. Dazu gehören der Trainings- und Validierungsverlust, die aktuelle Lernrate, die Epochendauer sowie der GPU-Speicherverbrauch. Zusätzlich werden die Gradientennorm und die Anzahl der durch Gradient Clipping begrenzten Updates erfasst, um die Stabilität des Trainings beurteilen zu können.

In regelmäßigen Abständen werden Samples generiert, um die visuelle Qualität der erzeugten Volumina im Trainingsverlauf zu überprüfen. Checkpoints werden an festgelegten Epochen gespeichert sowie für das Modell mit dem besten Validierungsverlust. Sämtliche Metriken werden nach jeder Epoche in einer JSON-Datei gesichert, sodass der Trainingsverlauf auch nachträglich vollständig analysiert werden kann.

6 Ergebnis

6.1 Trainingsergebnisse

Dieses Kapitel präsentiert die während des Trainings erfassten Metriken für drei Diffusionsmodelle unterschiedlicher Volumengröße und Konfiguration. Die Ergebnisse zeigen den Lernfortschritt über 300 Epochen anhand von Trainings- und Validierungsverlust.

6.1.1 Modellkonfigurationen

Alle Modelle teilen eine gemeinsame Basis-Architektur und Trainingsinfrastruktur, die nachfolgend erläutert wird.

Gemeinsame Hyperparameter und Techniken

- **Epochen:** 300 – Eine Epoche bezeichnet einen vollständigen Durchlauf des gesamten Trainingsdatensatzes. Nach 300 Epochen hat das Modell jeden Scan hunderte Male in verschiedenen Rauschzuständen gesehen.
- **Batch-Größe:** Gibt an, wie viele Trainingsvolumen gleichzeitig in den Grafikkartenspeicher geladen und verarbeitet werden. Größere Batches stabilisieren den Gradienten, benötigen aber mehr VRAM. Bei 32^3 wurden 10 Volumen pro Schritt verarbeitet; bei 48^3 musste die Batch-Größe auf 3 reduziert werden, da jedes Volumen $3.4\times$ mehr Speicher belegt ($48^3/32^3 \approx 3.4$).
- **Lernrate (Learning Rate):** Steuert, wie stark die Gewichte nach jedem Schritt angepasst werden. Zu große Lernraten führen zu Instabilität, zu kleine zu langsamem Lernen. Es wird ein Warmup von der initialen Rate auf den Peak von 2×10^{-4} verwendet, gefolgt von kosinus-annealing Abkühlung auf 0.
- **v-Prediction:** Anstatt direkt das Rauschen ϵ vorherzusagen (standard DDPM), sagt das Modell eine Kombination aus Signal und Rauschen voraus – die sogenannte *velocity* v . Dies führt zu numerisch stabileren Gradienten und besserer Qualität bei kleinen Rauschstärken.
- **Kosinus-Annealing Schedule (Noise Schedule):** Definiert, wie viel Rauschen bei jedem Diffusionsschritt hinzugefügt wird. Ein Kosinus-Verlauf sorgt für einen glatteren Übergang zwischen reinem Signal und reinem Rauschen im Vergleich zu einem linearen Schedule, was die Lernstabilität verbessert.
- **Min-SNR Weighting:** Eine Gewichtungsstrategie für den Trainingsloss. Diffusionsschritte mit sehr niedrigem oder sehr hohem Signal-Rausch-Verhältnis (SNR) wer-

den weniger stark gewichtet, da sie schwieriger zu lernen sind und den Gradienten destabilisieren können. Dies verbessert die Trainingseffizienz und Stabilität.

- **EMA (Exponential Moving Average):** Zusätzlich zu den eigentlichen Modellgewichten werden parallel gemittelte Gewichte mitgeführt: $\theta_{\text{EMA}} = \alpha \cdot \theta_{\text{EMA}} + (1 - \alpha) \cdot \theta$ mit Decay $\alpha = 0.9999$. Die EMA-Gewichte reagieren träger auf kurzfristige Schwankungen und liefern daher stabilere, qualitativ bessere Generierungsergebnisse. Für die Inferenz werden ausschließlich die EMA-Gewichte verwendet.
- **Optimizer (AdamW):** Adaptiver Optimierer, der individuelle Lernraten pro Parameter berechnet und zusätzlich L2-Regularisierung (Weight Decay) anwendet, um Überanpassung zu reduzieren.
- **Gradient Clipping:** Wenn Gradienten während des Backward Pass einen Schwellwert überschreiten, werden sie auf diesen begrenzt. Dies verhindert explodierende Gradienten, die das Training destabilisieren oder zum Absturz bringen können.

Experiment 1: 32³ Baseline

- **Volumengröße:** $32 \times 32 \times 32$ Voxel
- **Trainingsdauer:** 56h 2m 46s (56.05 GPU-Stunden)
- **Batch-Größe:** 10
- **Trainings-/Validierungssamples:** 1126 / 125
- **Beste Epoche:** 45 (niedrigster Validierungs-Loss)
- **Besonderheit:** Referenzkonfiguration ohne Augmentierung

Trainingsmetriken:

- Initialer Trainings-Loss (Epoch 0): 0.438
- Finaler Trainings-Loss (Epoch 299): 0.00051
- Minimaler Trainings-Loss: 0.00046
- Reduktion: 99.88%

Validierungsmetriken:

- Initialer Validierungs-Loss: 0.320
- Finaler Validierungs-Loss: 0.109
- Minimaler Validierungs-Loss: 0.0573 (beste Performance, Epoch 45)
- Reduktion: 65.91%

Experiment 2: 32³ Data Augmentation

- **Volumengröße:** 32 × 32 × 32 Voxel
- **Trainingsdauer:** 55h 47m 51s (55.80 GPU-Stunden)
- **Batch-Größe:** 10
- **Trainings-/Validierungssamples:** 1126 / 125
- **Beste Epoche:** 102 (niedrigster Validierungs-Loss)
- **Besonderheit:** On-the-fly zufällige links-rechts Flip-Augmentierung (p=0.5)

Trainingsmetriken:

- Initialer Trainings-Loss: 0.438
- Finaler Trainings-Loss: 0.00164
- Minimaler Trainings-Loss: 0.00156
- Reduktion: 99.63%

Validierungsmetriken:

- Initialer Validierungs-Loss: 0.353
- Finaler Validierungs-Loss: 0.104
- Minimaler Validierungs-Loss: 0.0612 (beste Performance, Epoch 102)
- Reduktion: 70.60%

Experiment 3: 48³ Baseline

- **Volumengröße:** 48 × 48 × 48 Voxel
- **Trainingsdauer:** 187h 24m 26s (187.41 GPU-Stunden)
- **Batch-Größe:** 3 (reduziert aufgrund höherer Speicheranforderungen)
- **Trainings-/Validierungssamples:** 1077 / 119
- **Beste Epoche:** 276 (direkt nach dem Loss-Spike bei Epoch 275)
- **Besonderheit:** Höhere räumliche Auflösung, keine Augmentierung

Trainingsmetriken:

- Initialer Trainings-Loss: 0.344
- Finaler Trainings-Loss: 0.0082
- Minimaler Trainings-Loss: 0.0046
- Reduktion: 97.62%

Validierungsmetriken:

- Initialer Validierungs-Loss: 0.214
- Finaler Validierungs-Loss: 0.0611
- Minimaler Validierungs-Loss: 0.0445 (beste Performance, Epoch 275)
- Reduktion: 71.46%

6.1.2 Wichtigste Erkenntnisse

1. Generalisierungsverhalten Der initiale Validierungs-Loss ist in allen Modellen deutlich niedriger als der Trainings-Loss (32³ Baseline: 0.32 vs. 0.44; 48³ Baseline: 0.21 vs. 0.34). Dies deutet darauf hin, dass die Trainings- und Validierungsverteilungen ähnlich sind und das Modell nicht stark überanpasst. Der Trainings-Loss sinkt kontinuierlich bis nahe Null, während der Validierungs-Loss ein Plateau erreicht, was typisches Verhalten für gut regulierte Diffusionsmodelle ist.

2. Auswirkung von Datenaugmentierung Die Data-Augmentation (32³ Baseline vs. 32³ DA) zeigt folgende Effekte:

- Trainings-Loss ist leicht höher (0.00051 vs. 0.00164), da die augmentierten Daten mehr Variabilität aufweisen
- Validierungs-Loss-Reduktion ist stärker (65.91% vs. 70.60%), was bessere Generalisierung andeutet
- Beste Epoche liegt später (45 vs. 102), was auf langsamere, aber robustere Konvergenz hindeutet
- Das 32³ DA Modell zeigt ab Epoche 132 erste Anzeichen von Überanpassung

Die Augmentierung verbessert die Robustheit des Modells durch Exposition gegenüber räumlichen Variationen und fördert das Erlernen invarianter Merkmale.

3. Effekt der Volumengröße Der Vergleich 32^3 vs. 48^3 zeigt:

- 48^3 trainiert $3.3\times$ länger (187.41 GPU-h vs. 56.05 GPU-h) aufgrund der höheren räumlichen Komplexität
- Batch-Größe musste von 10 auf 3 reduziert werden, um in den VRAM zu passen
- Initialer Loss ist niedriger (0.344 vs. 0.438 für Trainings-Loss), was auf effizientere Initialisierung hindeutet
- Finaler Validierungs-Loss ist besser (0.0445 vs. 0.0573), was zeigt, dass größere Volumen mit mehr Kontextinformation profitieren
- Die höhere Auflösung ermöglicht feinere anatomische Details in der Generierung und bessere räumliche Kohärenz

4. Lernkurven und Konvergenz Alle Modelle zeigen generell stabile, monoton fallende Lernkurven über 300 Epochen. Die exponentielle Abnahme des Trainings-Loss gegenüber der sublinearen Abnahme des Validierungs-Loss ist typisch für Diffusionsmodelle.

Anomalien - Loss Spikes:

- **32^3 DA, Epoch 4:** Früher Loss-Spike (Train: 0.0807, Val: 0.1883) zu Beginn des Trainings, danach stabile Konvergenz.
- **48^3 Baseline, Epoch 7:** Zweiter früher Spike (Train: 0.0708, Val: 0.1939), typisch für die Eingewöhnungsphase.
- **48^3 Baseline, Epoch 275:** Hauptanomalie – Trainings-Loss springt von 0.00457 auf 0.0322 (ca. 600% Anstieg). Paradoxerweise sinkt der Validierungs-Loss gleichzeitig auf sein globales Minimum (0.0445). Das Modell stabilisiert sich ab Epoch 276 vollständig.

Interpretation des Epoch-275-Ereignisses: Der Crash ist nicht kritisch. Das Modell erholt sich vollständig und erreicht danach sogar die beste Validierungsperformance (beste Epoche: 276). Dies könnte ein kritischer Reorganisationspunkt sein, an dem das Modell seinen internen Repräsentationsraum neu strukturiert. Keine NaN/Inf-Ereignisse wurden beim 32^3 -Modell registriert; beim 48^3 -Modell traten 5 isolierte NaN/Inf-Ereignisse auf, die jedoch keine unkontrollierte numerische Instabilität verursachten. Ähnliche Phänomene wurden in Deep Learning beobachtet, insbesondere bei großen 3D-Modellen mit komplexem Gradientenfluss.

6.1.3 Detaillierte Metriken-Analyse

Neben den Loss-Kurven wurden weitere kritische Trainings-Metriken analysiert:

Computational Efficiency - Timing Breakdown

Ein Trainingsschritt besteht aus drei Phasen, deren Zeitanteile gemessen wurden:

- **Forward Pass** bezeichnet die Vorwärtsberechnung: Das veräuschte Volumen wird durch alle Schichten des Netzwerks geleitet, um die vorhergesagte Velocity v zu berechnen. Dies entspricht reiner Modell-Inferenz und ist verhältnismäßig schnell ($\sim 1\text{--}2\%$ bei 32^3 , $\sim 2.5\%$ bei 48^3).
- **Backward Pass** berechnet die Gradienten durch Rückwärtspropagation (Backpropagation): Für jeden der Millionen Modellparameter wird bestimmt, wie stark er den Loss beeinflusst. Bei 3D-Architekturen mit vielen Schichten und 3D-Faltungen ist dies rechnerisch sehr aufwendig und dominiert die Trainingszeit (32^3 : $\sim 72\%$, 48^3 : $\sim 60\%$).
- **Data Loading** umfasst das Lesen der NIfTI-Volumina von Disk, Normalisierung und Rausch-Sampling. Der geringe Zeitanteil ($<1\%$) zeigt, dass die Datenpipeline nicht der Flaschenhals ist – die GPU wartet nicht auf Daten.

Der relativ niedrigere Backward-Pass-Anteil beim 48^3 -Modell (60% vs. 72%) erklärt sich durch den proportional größeren Forward Pass bei höherer räumlicher Auflösung.

Gradient & Stability Metrics

Stabilitäts-Indikatoren zeigen die numerische Robustheit während des Trainings:

- **Gradient Clipping Events:** Zählt, wie oft der Gradient in einem Trainingsschritt den definierten Schwellwert überschritt und begrenzt wurde. Hohe Werte (617 bei 32^3 Baseline, 888 bei 32^3 DA, 442 bei 48^3) sind bei Diffusionsmodellen normal und zeigen, dass der Mechanismus aktiv arbeitet. Die höhere Anzahl beim DA-Modell ist auf die größere Variabilität durch die Augmentierung zurückzuführen.
- **NaN/Inf Events:** Treten auf, wenn eine Berechnung numerisch undefiniert wird (z. B. Division durch Null oder Overflow). 0 Ereignisse bei den 32^3 -Modellen zeigen vollständige numerische Stabilität. Die 5 isolierten Ereignisse beim 48^3 -Modell führten zu keiner Destabilisierung – das Modell konnte sich selbst korrigieren.
- **Loss Spikes:** Plötzliche sprunghafte Anstiege des Loss über mehrere Standardabweichungen hinaus. 0 Spikes beim 32^3 Baseline bestätigen stabiles Training; 1 früher Spike beim 32^3 DA-Modell (Epoch 4) ist in der Anfangsphase mit Augmentierung typisch; die 2 Spikes beim 48^3 -Modell (Epochen 7 und 275) wurden vollständig überwunden.
- **Durchschnittliche Gradienten-Norm:** Maß für die Gesamt-Größe des Gradienten über alle Parameter. Ein konsistenter Wert ($\sim 0.19\text{--}0.24$) über das Training zeigt,

dass das Modell stabil lernt und weder zu aggressiv noch zu zaghaft aktualisiert wird. Werte nahe Null am Ende (~ 0.019 – 0.025) deuten auf Konvergenz hin.

- **EMA Divergence:** Misst den Abstand zwischen den aktuellen Modellgewichten und den EMA-Gewichten. Ein moderater, stabiler Wert (~ 45 – 49) zeigt, dass die EMA-Gewichte dem Training folgen, ohne zu stark zu divergieren – ein Zeichen für gesunde Lernfortschritte.

Das Training war insgesamt stabil mit nur isolierten, vollständig überwundenen Anomalien.

Ressourcennutzung

- **32³ Modelle:** ~ 56 GPU-Stunden, Peak Memory ~ 10.7 GB (91% der GTX 1080 Ti VRAM), Batch-Größe 10
- **48³ Modell:** ~ 187 GPU-Stunden, Peak Memory ~ 10.1 GB, Batch-Größe 3 (durch VRAM-Beschränkung)
- **Speicherauslastung:** Alle Modelle nutzen den verfügbaren VRAM (11.7 GB) nahezu vollständig aus

6.1.4 Fazit

Die Trainingsergebnisse demonstrieren, dass alle drei Konfigurationen erfolgreich trainieren und stabile Lernkurven aufweisen. Die 48³-Auflösung bietet sowohl bessere Validierungsmetriken als auch höhere Präzision für medizinische Bildgenerierung, rechtfertigt jedoch die $3.3\times$ längere Trainingsdauer und erfordert eine deutlich reduzierte Batch-Größe (3 statt 10). Datenaugmentierung verbessert die Generalisierung messbar, führt aber zu späterer Konvergenz (Epoche 102 statt 45) und ersten Anzeichen von Überanpassung ab Epoche 132. Alle Loss-Spikes wurden vollständig überwunden, wobei der Spike bei 48³ in Epoch 275 paradoxerweise zum besten Validierungsergebnis führte.

Visualisierungen

Zwei zentrale Visualisierungen zeigen die Trainings-Performance:

1. **Loss Curves (Abbildung 01):** Zeigt den Verlauf von Trainings- (blau, solid) und Validierungs-Loss (orange, gestrichelt) über 300 Epochen für alle drei Modelle. Deutlich sichtbar sind die Loss-Spikes, insbesondere bei 48³ in Epoch 275.
2. **Training Time (Abbildung 05):** Horizontales Balken-Diagramm der GPU-Stunden zeigt die drastisch höhere Trainingszeit des 48³ Modells (187h vs. 56h für 32³).

Diese Visualisierungen dienen als direkte Referenzen für die oben beschriebenen analytischen Ergebnisse.

7 Schlussfolgerung

7.1 Fazit

7.2 Ausblick

Literatur

- [1] Sridaran Rajagopal u. a., Hrsg. *Artificial Intelligence Based Smart and Secured Applications: 4th International Conference, ASCIS 2025, Gujarat, India, September 11-13, 2025, Revised Selected Papers, Part III*. en. Bd. 2821. Communications in Computer and Information Science. Cham: Springer Nature Switzerland, 2026. ISBN: 978-3-032-17839-8 978-3-032-17840-4. DOI: 10.1007/978-3-032-17840-4. URL: <https://link.springer.com/10.1007/978-3-032-17840-4> (besucht am 18.02.2026).
- [2] Ahmed Elhadad, Mona Jamjoom und Hussein Abulkasim. „Reduction of NIFTI files storage and compression to facilitate telemedicine services based on quantization hiding of downsampling approach“. en. In: *Scientific Reports* 14.1 (März 2024), S. 5168. ISSN: 2045-2322. DOI: 10.1038/s41598-024-54820-4. URL: <https://www.nature.com/articles/s41598-024-54820-4> (besucht am 18.02.2026).
- [3] Department of Computer Applications, Kalasalingam Academy of Research and Education (Deemed to be University), Krishnankoil, Tamil Nadu, India. u. a. „An Medical Image File Formats and Digital Image Conversion“. en. In: *International Journal of Engineering and Advanced Technology* 9.1s4 (Dez. 2019), S. 74–78. ISSN: 22498958. DOI: 10.35940/ijeat.A1093.1291S419. URL: <https://www.ijeat.org/portfolio-item/A10931291S419/> (besucht am 18.02.2026).
- [4] Harald Nahrstedt. *Algorithmen für Ingenieure*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2018. ISBN: 978-3-658-19298-3 978-3-658-19299-0. DOI: 10.1007/978-3-658-19299-0. URL: <http://link.springer.com/10.1007/978-3-658-19299-0> (besucht am 10.10.2025).
- [5] Zhen LLeo" Liu. *Artificial Intelligence for Engineers: Basics and Implementations*. en. Cham: Springer Nature Switzerland, 2025. ISBN: 978-3-031-75952-9 978-3-031-75953-6. DOI: 10.1007/978-3-031-75953-6. URL: <https://link.springer.com/10.1007/978-3-031-75953-6> (besucht am 06.10.2025).
- [6] Andreas Kohne u. a. *Chatbots: Aufbau und Anwendungsmöglichkeiten von autonomen Sprachassistenten*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2020. ISBN: 978-3-658-28848-8 978-3-658-28849-5. DOI: 10.1007/978-3-658-28849-5. URL: <http://link.springer.com/10.1007/978-3-658-28849-5> (besucht am 06.10.2025).
- [7] Vasant Honavar. „Artificial Intelligence: An Overview“. en. In: (). (Besucht am 07.10.2025).
- [8] Stuart J. Russell und Peter Norvig. *Artificial intelligence: a modern approach*. en. Third edition, Global edition. Prentice Hall series in artificial intelligence. Boston

- Columbus Indianapolis: Pearson, 2016. ISBN: 978-0-13-604259-4 978-1-292-15397-1. (Besucht am 07. 10. 2025).
- [9] Paolo Bory, Simone Natale und Christian Katzenbach. „Strong and weak AI narratives: an analytical framework“. en. In: *AI & SOCIETY* 40.4 (Apr. 2025), S. 2107–2117. ISSN: 0951-5666, 1435-5655. DOI: 10 . 1007 / s00146 - 024 - 02087 - 8. URL: <https://link.springer.com/10.1007/s00146-024-02087-8> (besucht am 14. 10. 2025).
- [10] Carsten Lanquillon und Sigurd Schacht. *Knowledge Science - Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2023. ISBN: 978-3-658-41688-1 978-3-658-41689-8. DOI: 10 . 1007 / 978 - 3 - 658 - 41689 - 8. URL: <https://link.springer.com/10.1007/978-3-658-41689-8> (besucht am 06. 10. 2025).
- [11] Robert Ciesla. *The Book of Chatbots: From ELIZA to ChatGPT*. en. Cham: Springer Nature Switzerland, 2024. ISBN: 978-3-031-51003-8 978-3-031-51004-5. DOI: 10 . 1007 / 978 - 3 - 031 - 51004 - 5. URL: <https://link.springer.com/10.1007/978-3-031-51004-5> (besucht am 14. 10. 2025).
- [12] Stefan Feuerriegel u. a. „Generative AI“. en. In: *Business & Information Systems Engineering* 66.1 (Feb. 2024), S. 111–126. ISSN: 2363-7005, 1867-0202. DOI: 10 . 1007 / s12599 - 023 - 00834 - 7. URL: <https://link.springer.com/10.1007/s12599-023-00834-7> (besucht am 14. 02. 2026).
- [13] Chieh-Hsin Lai u. a. *The Principles of Diffusion Models*. en. arXiv:2510.21890 [cs]. Okt. 2025. DOI: 10 . 48550 / arXiv . 2510 . 21890. URL: <http://arxiv.org/abs/2510.21890> (besucht am 14. 02. 2026).
- [14] Lars Ruthotto und Eldad Haber. „An introduction to deep generative modeling“. en. In: *GAMM-Mitteilungen* 44.2 (Juni 2021), e202100008. ISSN: 0936-7195, 1522-2608. DOI: 10 . 1002 / gamm . 202100008. URL: <https://onlinelibrary.wiley.com/doi/10.1002/gamm.202100008> (besucht am 15. 02. 2026).
- [15] Takeshi Okadome. *Essentials of Generative AI*. en. Singapore: Springer Nature Singapore, 2025. ISBN: 978-981-96-0028-1 978-981-96-0029-8. DOI: 10 . 1007 / 978 -

- 981 - 96 - 0029 - 8. URL: <https://link.springer.com/10.1007/978-981-96-0029-8> (besucht am 15.02.2026).
- [16] Jonathan Ho, Ajay Jain und Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. en. arXiv:2006.11239 [cs]. Dez. 2020. DOI: 10.48550/arXiv.2006.11239. URL: <http://arxiv.org/abs/2006.11239> (besucht am 15.02.2026).
- [17] Christopher M. Bishop. *Pattern recognition and machine learning*. en. Information science and statistics. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [18] Alex Nichol und Prafulla Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. en. arXiv:2102.09672 [cs]. Feb. 2021. DOI: 10.48550/arXiv.2102.09672. URL: <http://arxiv.org/abs/2102.09672> (besucht am 17.03.2026).
- [19] Jiaming Song, Chenlin Meng und Stefano Ermon. *Denoising Diffusion Implicit Models*. en. arXiv:2010.02502 [cs]. Okt. 2022. DOI: 10.48550/arXiv.2010.02502. URL: <http://arxiv.org/abs/2010.02502> (besucht am 19.03.2026).
- [20] Keiron O'Shea und Ryan Nash. *An Introduction to Convolutional Neural Networks*. en. arXiv:1511.08458 [cs]. Dez. 2015. DOI: 10.48550/arXiv.1511.08458. URL: <http://arxiv.org/abs/1511.08458> (besucht am 04.03.2026).
- [21] Nhat-Duc Hoang und Van-Duc Tran. „Deep Learning-Based Predictive Modeling of Urban Heat Stress Using Transformer Network, Deep Neural Network, and Convolutional Neural Network“. en. In: *Remote Sensing in Earth Systems Sciences* 8.4 (Dez. 2025), S. 1161–1184. ISSN: 2520-8195, 2520-8209. DOI: 10.1007/s41976-025-00242-3. URL: <https://link.springer.com/10.1007/s41976-025-00242-3> (besucht am 04.03.2026).
- [22] Wei Shen u. a. *Hands-On Computer Vision*. en. Singapore: Springer Nature Singapore, 2026. ISBN: 9789819548347 9789819548354. DOI: 10.1007/978-981-95-4835-4. URL: <https://link.springer.com/10.1007/978-981-95-4835-4> (besucht am 06.03.2026).
- [23] Alex Krizhevsky, Ilya Sutskever und Geoffrey E. Hinton. „ImageNet classification with deep convolutional neural networks“. en. In: *Communications of the ACM* 60.6 (Mai 2017), S. 84–90. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3065386. URL: <https://dl.acm.org/doi/10.1145/3065386> (besucht am 04.03.2026).
- [24] Marvin John Ignacio u. a. „Revisiting U-Net: a foundational backbone for modern generative AI“. en. In: *Artificial Intelligence Review* 59.2 (Nov. 2025), S. 45. ISSN: 1573-7462. DOI: 10.1007/s10462-025-11450-0. URL: <https://link.springer.com/10.1007/s10462-025-11450-0> (besucht am 24.02.2026).
- [25] Olaf Ronneberger, Philipp Fischer und Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. en. arXiv:1505.04597 [cs]. Mai 2015. DOI: 10.48550/arXiv.1505.04597. URL: <http://arxiv.org/abs/1505.04597> (besucht am 22.02.2026).
- [26] Pinar Civicioglu und Erkan Besdok. „Image discrimination robustness of classical image quality metrics: an analysis on MAE, MSE, ERGAS, LMSE, SSIM, SAM, UIQI, PSNR, NIQE, PIQE, SNR, VIF, FSIM, GMSD, and NCC“. en. In: *Quality & Quantity* (Okt. 2025). ISSN: 0033-5177, 1573-7845. DOI: 10.1007/s11135-025-02404-3.

- URL: <https://link.springer.com/10.1007/s11135-025-02404-3> (besucht am 01.03.2026).
- [27] Martin Heusel u. a. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. en. arXiv:1706.08500 [cs]. Jan. 2018. DOI: 10.48550/arXiv.1706.08500. URL: <http://arxiv.org/abs/1706.08500> (besucht am 01.03.2026).
 - [28] Christian Szegedy u. a. *Rethinking the Inception Architecture for Computer Vision*. en. arXiv:1512.00567 [cs]. Dez. 2015. DOI: 10.48550/arXiv.1512.00567. URL: <http://arxiv.org/abs/1512.00567> (besucht am 01.03.2026).
 - [29] Peng Long und Xiaozhou Guo. *Generative Adversarial Network: Principle and Practice*. en. Singapore: Springer Nature Singapore, 2026. ISBN: 978-981-96-9403-7 978-981-96-9404-4. DOI: 10.1007/978-981-96-9404-4. URL: <https://link.springer.com/10.1007/978-981-96-9404-4> (besucht am 01.03.2026).
 - [30] D. C. Dowson und B. V. Landau. „The Fréchet Distance between Multivariate Normal Distributions“. In: *Journal of Multivariate Analysis* 12.3 (1982), S. 450–455. ISSN: 0047-259X. DOI: 10.1016/0047-259X(82)90077-X.
 - [31] Sho Maruyama. „Properties of the SSIM metric in medical image assessment: correspondence between measurements and the spatial frequency spectrum“. en. In: *Physical and Engineering Sciences in Medicine* 46.3 (Sep. 2023), S. 1131–1141. ISSN: 2662-4729, 2662-4737. DOI: 10.1007/s13246-023-01280-1. URL: <https://link.springer.com/10.1007/s13246-023-01280-1> (besucht am 02.03.2026).
 - [32] Zhou Wang u. a. „Image quality assessment: from error visibility to structural similarity“. en. In: *IEEE Transactions on Image Processing* 13.4 (Apr. 2004), S. 600–612. ISSN: 1057-7149, 1941-0042. DOI: 10.1109/TIP.2003.819861. URL: <https://ieeexplore.ieee.org/document/1284395/> (besucht am 02.03.2026).
 - [33] Jim Nilsson und Tomas Akenine-Möller. *Understanding SSIM*. en. arXiv:2006.13846 [eess]. Juni 2020. DOI: 10.48550/arXiv.2006.13846. URL: <http://arxiv.org/abs/2006.13846> (besucht am 02.03.2026).
 - [34] GeeksforGeeks. *Why is Python the Best-Suited Programming Language for Machine Learning?* <https://www.geeksforgeeks.org/blogs/why-is-python-the-best-suited-programming-language-for-machine-learning/>. Besucht am: 15.03.2026. Aug. 2025.
 - [35] Alicia Durrer, Philippe C. Cattin und Julia Wolleb. *Denoising Diffusion Models for Inpainting of Healthy Brain Tissue*. en. arXiv:2402.17307 [eess]. Okt. 2024. DOI: 10.48550/arXiv.2402.17307. URL: <http://arxiv.org/abs/2402.17307> (besucht am 15.03.2026).

A Anhang A

Weitere Abbildungen